

Prompting Cheat Sheet

Built for Claude models, but may include standard prompting engineering guidelines that can be applied with other models. Many of these are variations of recommendations from [Claude Best Practices](#)

Prompting is the single best way to influence the output of an LLM. Here are Prompt Engineering principles to consider when constructing a prompt.

Provide Specific Instructions

Claude can't read your mind. Whenever possible, provide details relevant to the task to Claude.

Base: Review the code

Better: Conduct a review of the new Login Feature. Consider both security risks and user experience.

Base: Fix the bug with the new login UI

Better: When using the new login UI, the submit request does not seem to go through. Double check that the submit request is being handled properly so the page reacts correctly.

Spec-driven Development

For large scale or complex tasks, consider using a spec to help guide development. This could be a Concept of Operations, derived from assignment instructions, or manually generated to give specific form to the piece of software you are trying to develop.

Base: Add a new login page to our website

Better: Use the login section in @website-spec to implement the login page

Give Claude the tools to check itself (and encourage their use)

Along with developing tests with your project, allowing Claude to check its own work will produce more reliable code.

Base: Add a new login page to our website

Better: Add a new login page to our website. Use the unit tests to check your implementation. Use the website tools at your disposal (playwright mcp) to verify the UI of the page.

Instead of restricting Claude, Promote positive behaviors

By encouraging desirable behavior, you may observe a more natural and coherent output. While there are still times to discourage behavior, encouraging other behaviors will allow the LLM to still exercise its full ability to accomplish its task.

Base: Don't use bulleted lists in the code report

Better: Strive to use complete paragraphs that flow together when writing the code report

Point Claude to existing patterns

Rather than describing a feature from scratch, point Claude at code that already solves a similar problem. This keeps new work consistent with your codebase and encourages reuse over reinvention.

Base: Add a calendar widget to the dashboard

Better: Look at how the existing widgets on the dashboard are built (@HotDogWidget is a good example). Follow that same pattern to add a calendar widget, using only the libraries already in the project.